QCon
全球软件开发大会

InfoQ 技术大会

把 Loop 接进团队：游戏研发实践中的 Loop Engineering

任磊达

腾讯高级后台开发工程师

QCon 上海 · Loop Engineering



Loop 是什么？

A. OpenClaw 定时任务

B. Anthropic 骗人烧 token 的新叙事

C. Harness / Hook / CI 这些老瓶装新酒

D. Chatbot 进化成 Agent 的核心科技

E. 《卷》

个人介绍

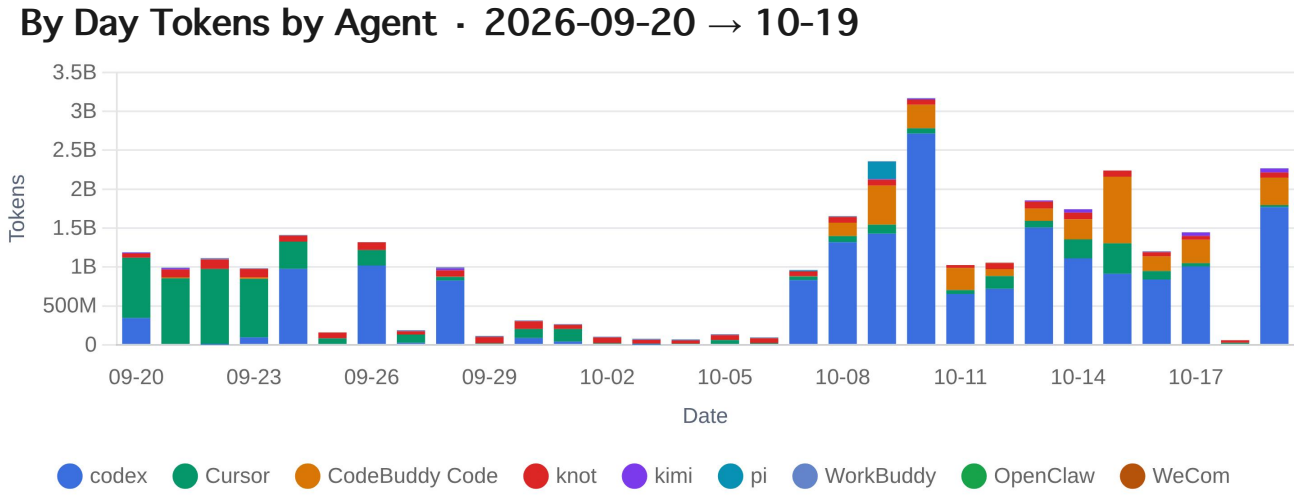


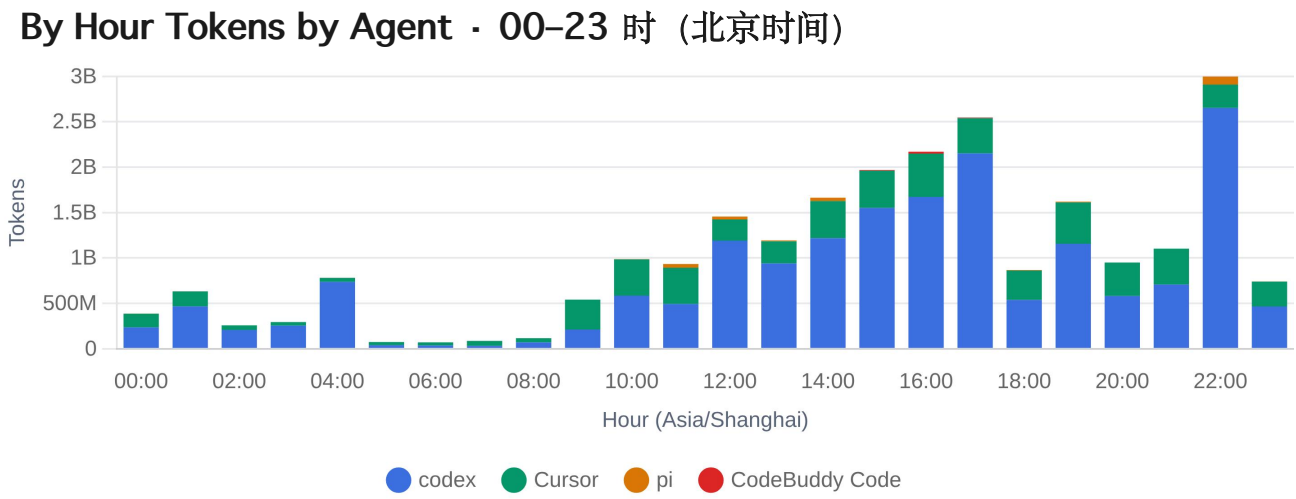
任磊达

腾讯高级后台开发工程师

- 30 天消耗 300 亿 token 的 AI Builder
- 推动百人游戏研发团队 Agentic 转型

微信扫码交流





QCon
全球软件开发大会

InfoQ 技术大会

目录

01 背景：Vibe Coding 了一个游戏 Demo，然后呢

02 Loop Engineering 实践中的暴论

03 实践：Loop Harness 让策划自己开发需求

04 实践：工作台让个人 Loop 到团队 Loop

01

背景

Harness 是探索可能，Loop 是锚定价值

很多 Vibe Coding 的场景都在游戏有很好的落地



Introducing Playground：用文字就能做游戏



来源：Google Blog · 2026-10-07 · https://blog.google/innovation-and-ai/technology/ai/playground-experimental-gaming-platform/

同样是赛车 Demo，半年的差距



3 月 · Cursor + Opus 4.5



10 月 · Grok Bot

Demo：radaren.github.io/sands-kart-demo

当模型进化推动 Demo 进化的时候，我们的价值在哪里？

从 Demo 到长青游戏



从 Demo 到长青：反馈 · 方向 · 责任

缺 Loop 的完美 Demo，很难熬过试点炼狱

Demo：即兴 vibe

一句话生成、试玩、改一版再试。
上下文短、责任轻、可丢弃。

证明了「能做出来」，适合探索。

Demo · 探索期

长青游戏：活过版本

需求变更、联调、回归、热更、多人接手。

难的不只是写得快——
而是下一版往哪走、什么算过线。

长青 · 长期运营

同一层面：Harness 有没有积极反馈，决定能不能 Loop 出长期生产价值

OpenClaw

浅尝辄止，慢慢淡出视野
没有积极反馈 → 难 Loop 出长期生产价值

浅尝辄止

GrokBot

迭代完整
与 harness 一起迭代

迭代完整 · 与 Harness 一起

从 ShowCase 到 Engineering

从 Pass@K 到 Pass^K

Pass@K 好看，为什么现场仍交不出信任？

抽奖式生成：多抽几次碰绿，单次偶然对上 ≠ 下一圈还信。

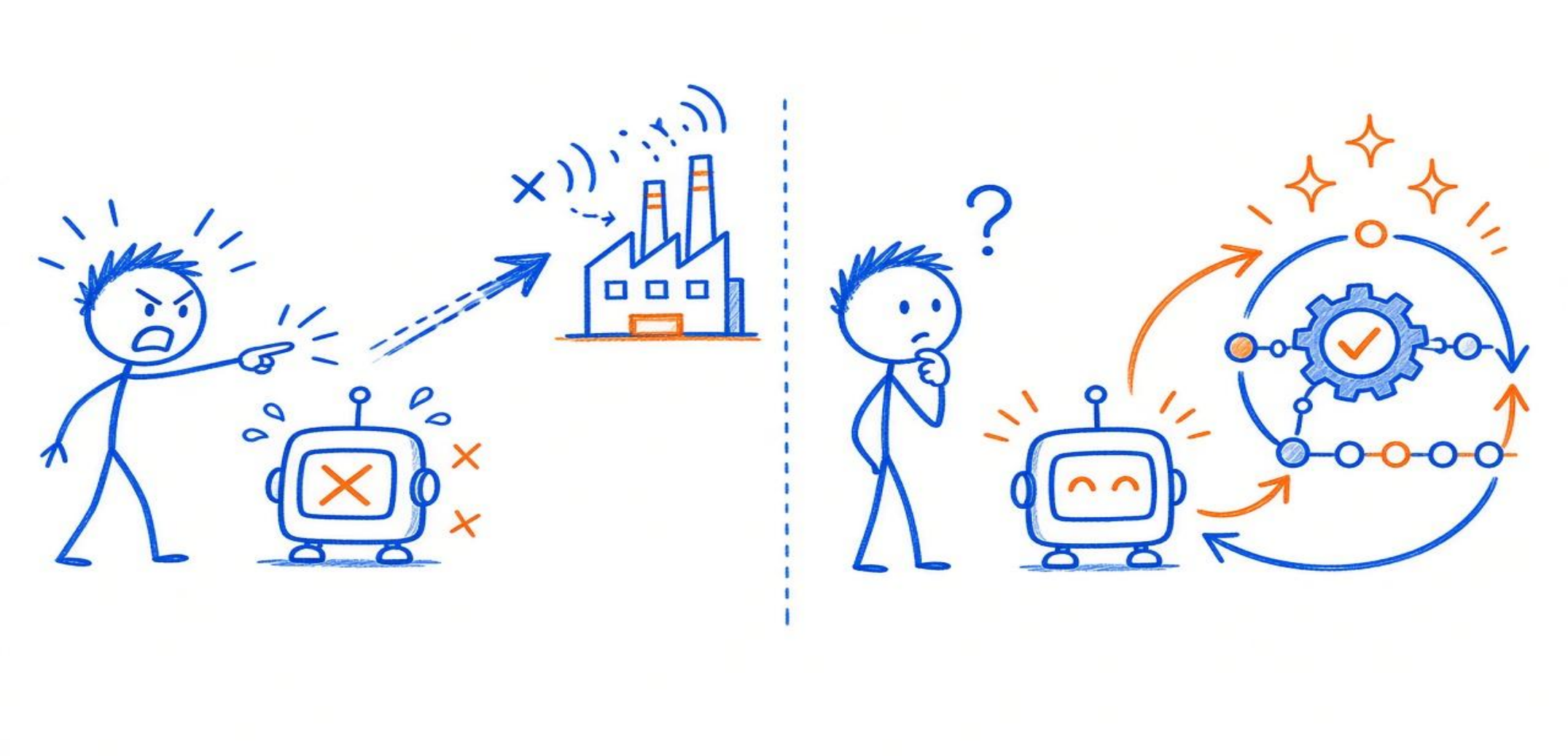
Pass^K：每一步可验收、可复利——信任是连乘，不是并集抽奖。（面向概率 → 面向进化）

疲惫感 = 动机：重采、Pass@K、人在场——仍交不出「这一次过了」。

信号是：已经在为目标做减法，只是还没有 Loop。

抽奖碰绿交不出信任；没反馈的组件也交不出长期价值——都指向同一件事：把反馈接进 Loop

Prompt Agent 还是 Build Loop?



Prompt Agent vs Build Loop · 对比

Claude Code 作者 Boris（约 6 月）谈到：自己已经不在做 prompt engineering，而是在 build loop。
agent 出错时，你在纠正谁？反馈信号最终进了哪条回路？

Prompt Agent

agent 出错时：改 prompt、换说法，再试一次。

纠正的是这一次的行为——
修好了，下一轮还可能重来。

改 prompt · 再跑一次

Build Loop

更像 PUA 它：不只直接帮它修，
而是问——你为什么出错？下次能不能不出错？

让它从成型的视角 review 现有工作流。
「为什么出错、能不能不再出错」变得更重要。

问原因 · 更新工作流

思考复利：纠一次 → 沉淀进工作流 → 下次少犯
反馈只到厂商、工作流收不到——就越用越好不了

反馈信号流向

为什么常规交流让 agent「不能越用越好用」

常见路径

用户纠正 → 模型厂商 / Agent 开发者

工作流本身收不到信号。
下次同一类错，还得再改一次 prompt。

信号出工作流

Build Loop 路径

用户纠正 → 工作流（规则 / Skill / 验收）

纠一次，沉淀一次。
下一次同一类错，少犯或不犯。

信号进工作流 · 思考复利

思考复利：纠一次 → 沉淀进工作流 → 下次少犯
反馈只到厂商、工作流收不到——就越用越好不了

Loop Engineering 实践之前，今年的一些热词



你们 Agent 提效到底有多少啊?

那得看需要多少 control，如果不需要 control，可以提效十倍。

02

Loop Engineering 实践中的暴论

纠错如何复利 · 过线如何工程化

Q1 为什么我不给项目搭建「知识库」?

Q2 为什么我不让 Agent 写单元测试?

Q3 为什么不给 Skill 做 Evaluation?

Q4 为什么不用传统的 Git PR 工作流?

work loop：把人的注意力切成三种模式

Clarify 过早，会冒出大量自己也不知道的问题——
所以事前只定边界，执行交给 Agent，事后再判断。

事前 | 决策模式

输入：模糊需求

Agent clarify 边界、约束、验收标准与风险；人只做选择和拍板。

过早 Clarify：边界没摸到就问，问出来的很多是「我其实也不知道」。

输出：proposal.yaml / plan.yaml

→

事中 | 释放模式

输入：结构化计划

Agent 按 DAG、状态机执行；hook 盯确定性问题；人不盯屏幕。

输出：可审查的 MR

→

事后 | 审查模式

输入：MR 与执行记录

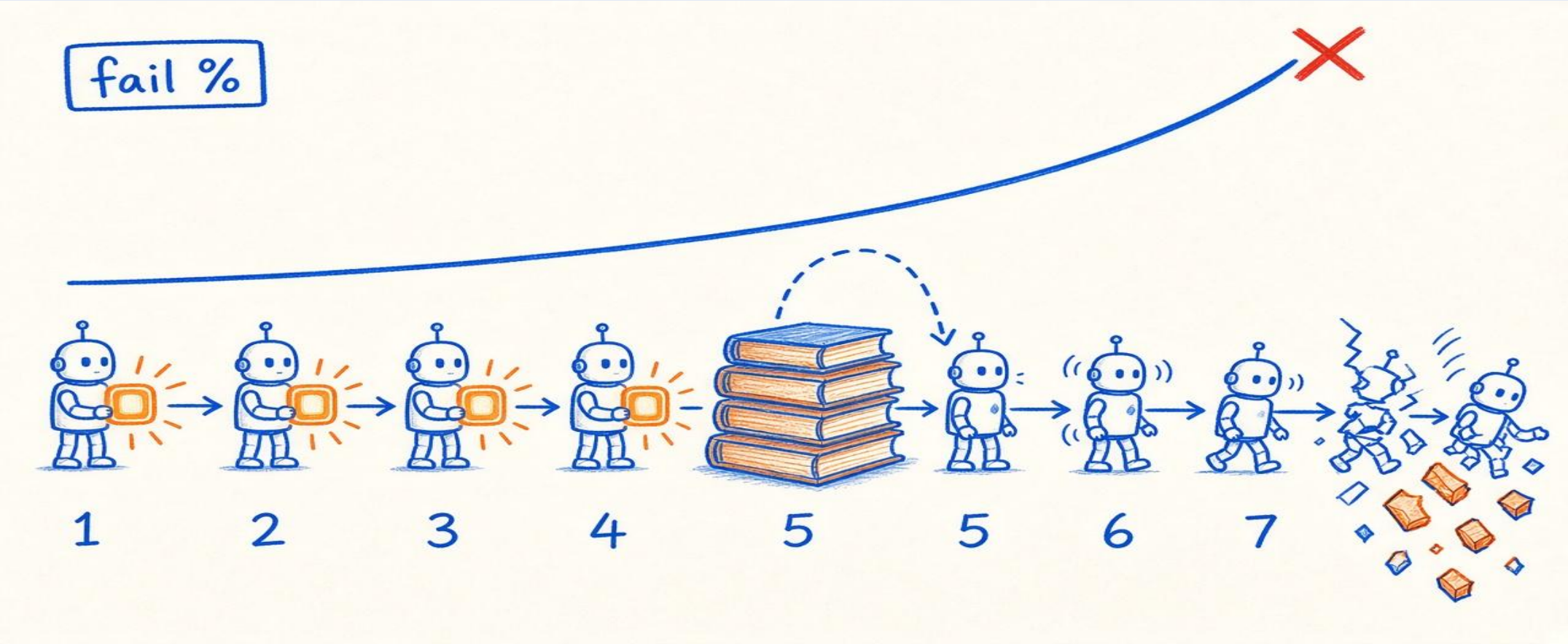
人判断结果是否正确；自动回灌 Hook / Skill。

输出：更强的 Harness

核心纪律：不要一直看着 Agent 执行，Review 放在 MR，Test 放在 Hook/Cron，执行交给 Agent。

> 人的注意力只在两个高价值节点出现：事前定边界，事后做判断。

为什么我不给项目搭建「知识库」？



agent 链路越长，失败率越快堆高；知识库拉长链路

知识库：大而全 · 无 owner · 只翻译 What

链路拉长只是症状——知识库还常变成无主的搜索噪音。

问题 · 大而全

无 owner

问题 · 只翻译 What

· Agent 变强后，知识库只加搜索链路，
· 没有实际场景就解决不了实际问题。
· 快速落地往往只翻译了代码的 What，
· 没给 Agent 新上下文。

无新上下文

往《活文档》看

· 好代码文档少，
· 靠行内注释就近呈现 Why。
· 对照好文档 vs 坏文档；
· 就近 8 层结构。

就近

· 知识库能很快证明 AI 工作量，
· 但大而全、与真实需求脱节。
· 缺乏真实 owner——
· 没人对准确性负责。

没人对准确性负责的知识库，只是搜索噪音

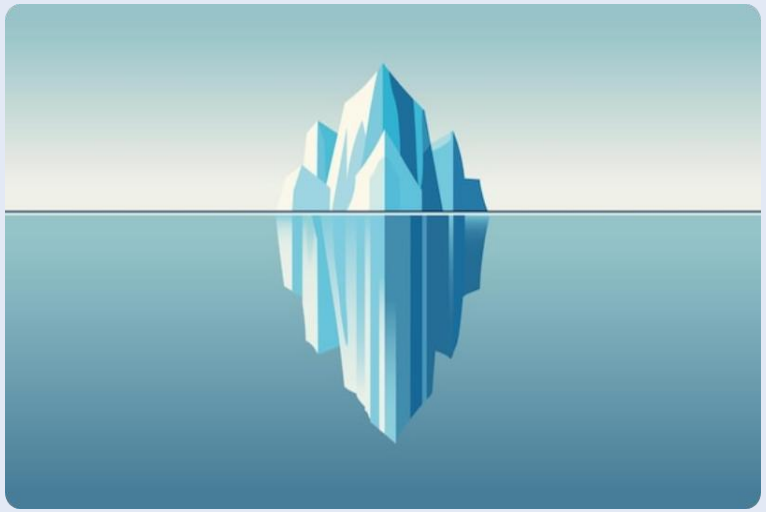
明辨是非：好文档 vs 坏文档

对比维度	好文档 (Good Doc)	坏文档 (Bad Doc)
目的	传播深思熟虑的知识	掩盖系统或流程的缺陷
本质	工作过程的自然产物	昂贵的沟通替代品
内容	关注高层级决策与逻辑	充斥低层级操作细节
价值	提升团队理解与共识	制造维护负担与噪音
时机	问题解决后的总结沉淀	问题解决前的强制要求
启示	促使流程与工具改进	粉饰太平，维持现状
知识管理视角	知识工程的自然产物；关注决策逻辑而非底层细节；在真实 Loop 中自动沉淀；有启发性	为完成任务而建的知识库，更像陈述性堆积，并不能解决问题

好文档是工作过程的自然产物，不是掩盖缺陷的替代品

就近文档 — 8层结构体系

Code is Cheap, Show me the talk



核心原则 · 就近原则

文档离代码越近，越容易保持同步。
Proximity Principle

8	项目文档	CODEBUDDY.md
7	跨模块文档	openspec/specs/
6	包文档	doc.go
5	边车文件	PURPOSE.md / mermaid.mmd
4	文件注释	Header
3	函数注释	Godoc
2	行内注释	Inline
1	命名描述	Self-explanatory

能用 10 句话描述清晰的系统，一定比要用 100 句话才能描述清晰的系统要好。

解法对比 — 大而全知识库 vs 每人一块模块 Skill

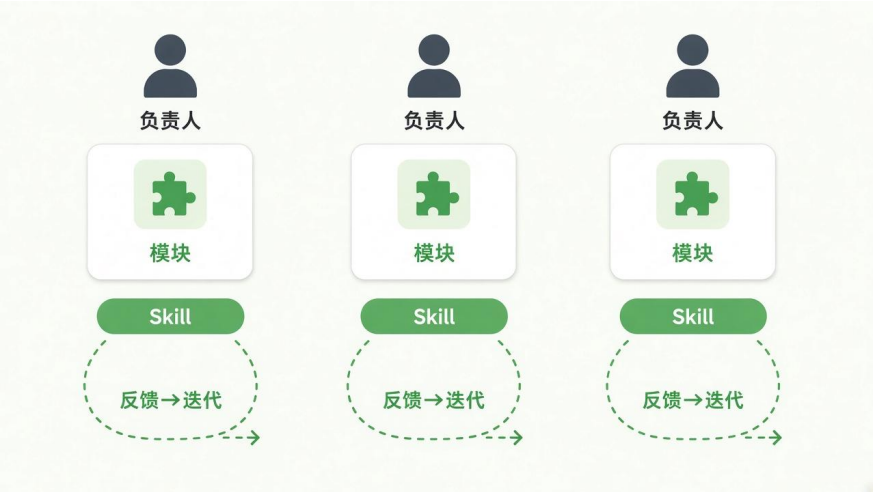
繁杂的知识库



大而全 · 无 owner · 只翻 What

无主 · 易过期

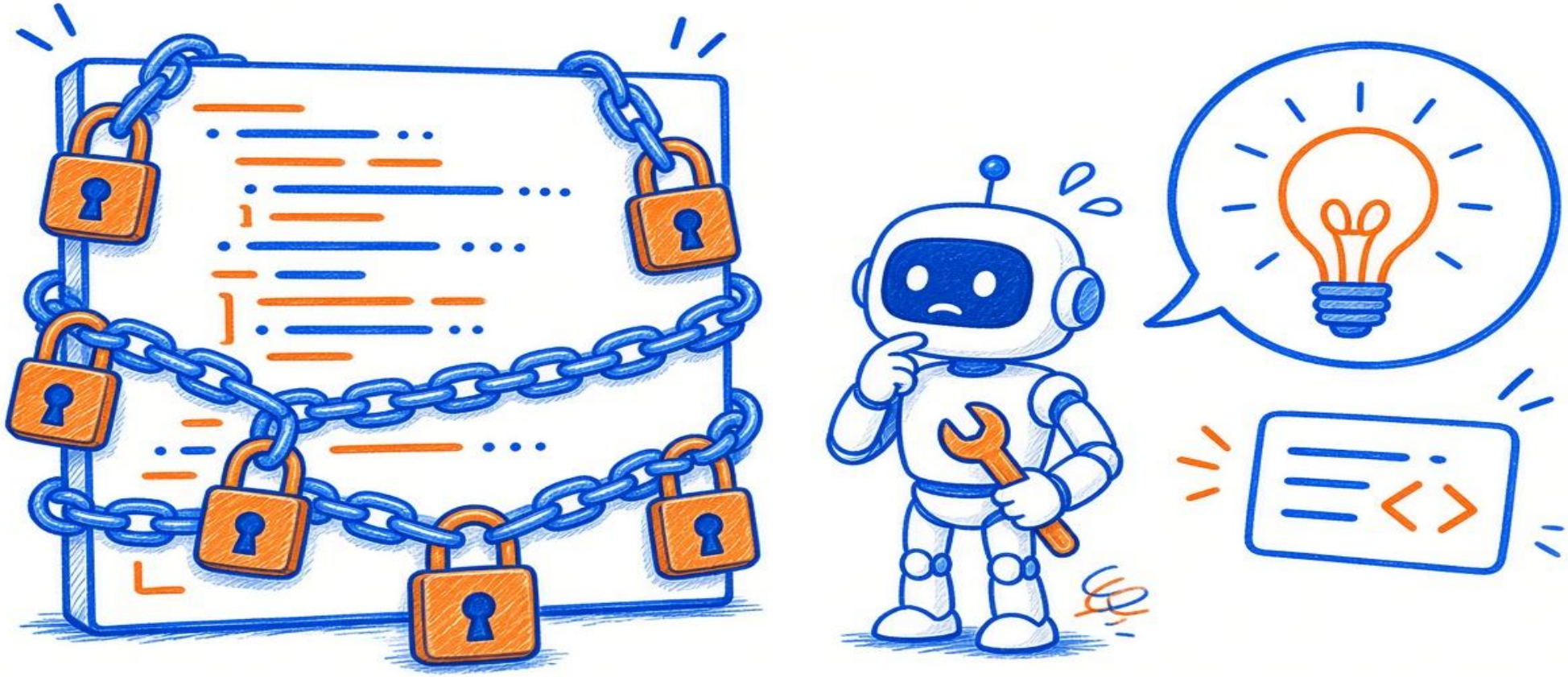
模块 Skill · 有 Owner



每个模块一个 Skill
负责人基于外部反馈精心迭代

有主 · 精迭代

为什么我不让 Agent 写单元测试?



单元测试：形式化 · 一把锁 · Human in the loop

问题 · 形式化

· 单元测试容易沦为形式化验证、照本宣科。

· 没有从其他角度为项目补充信息。

照本宣科

问题 · 一把锁

· 更像一把锁。

· 增加 token 消耗，让 Agent 更难改动逻辑。

· 逻辑变更时：又难改、又不敢改。

锁住改动

我的做法

· 把 What 归还给代码。

· Why 提供给行内注释。

· 验证留给 human in the loop。

Human in the loop

让 human 能 in the loop——验证不交给 Agent 自写自测

解法对比 — 覆盖率指标游戏 vs 责任块里的验证循环

100% 覆盖率 · 排行榜



100% 覆盖率 · 排行榜 · 冠军

指标好看 · 未必过线

责任块 · 集成测试 · 验证循环



What 归代码、Why 归注释
验证：集成测试 + 验证循环

责任块内闭环验证

为什么不给 Skill 做 Evaluation？

模型按周迭代，Harness 是资产还是债务？



Q3 Skill Eval：追随厂商 · 仍像抽奖 · 人负责

ROI 摆不正

· 初期用 Anthropic skill-creator eval 试过。

· 模型+Harness 上百个、按周迭代，

· eval 只会花大精力追随厂商。

· prompt 与模型强绑定——换模就失效。

· Evaluation 焊死某代模型 → 可能负优化。

追随厂商

ROI 摆不正

· 对模型本就有心理预期，不追求 100%。

· 多数 skill eval 不适配 AGENTS.md 与当前对话；

· eval 出 100% 能唤醒，真实场景仍像抽奖。

仍像抽奖

解法 · Loop

· **通过 Loop + 错误日记，**

· 解决 skill 不好用的场景。

· 人对 skill 负责——不可信就退化成 command。

· 在当前需求场景自然越用越好，

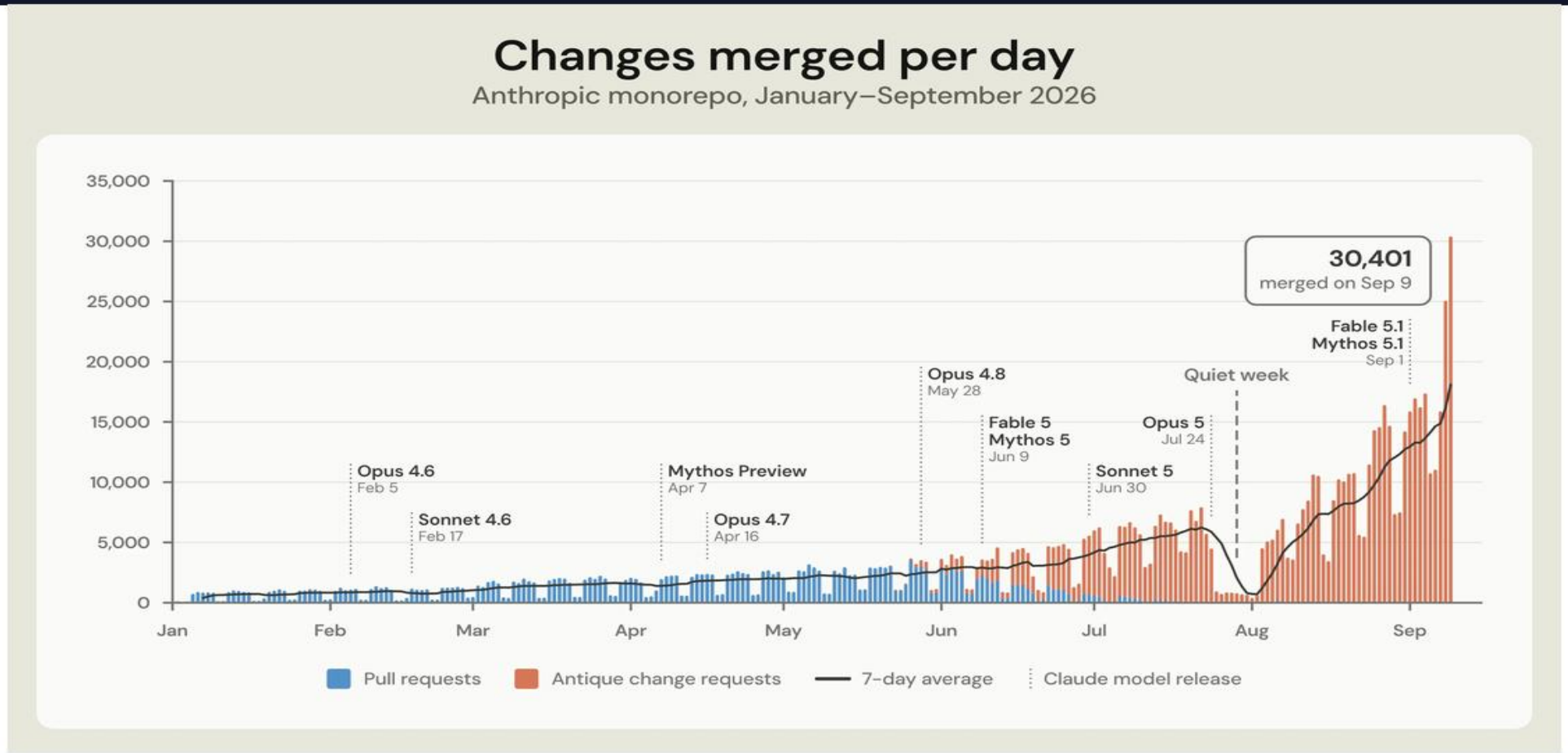
· 而非追求理论数值。

人负责

人对 skill 负责，比追求 eval 数值更重要

为什么不用传统的 Git PR 工作流？

Git / MR（蓝） vs Anthropic Antique（橙） · 日合入从 ~2.5k 到 3万+



来源：Anthropic · Redesigning source control · 2026-10-13 · Scion 替代 Git；Antique 替代传统 PR

MR：转嫁负担 · 负责人合入 · 对变更负责

问题

- Diff 太大带来团队沟通情绪。
- 开发把 AI 代码推送、自动生成 MR，
- 等于把模块工作量转嫁给团队——
- 而他人对模块不如负责人熟悉。

转嫁负担

解法

- ① 代码以 Agent 名义提交，
- 负责人自己合入 MR。
- ② 迭代 Review 基础设施：
- AI 预审（多 agent；审查角度基于外网问题 loop 迭代），
- 并让 agent 对 MR 提问题。

负责人合入

收束

- **MR 不再是审批流程，**
- 而是确保负责人
- 对变更负责的流程。

对变更负责

MR 不是审批——是确保负责人对变更负责

问题链收束：负责，才把 Loop 接下去

Q1 为什么我不给项目搭建「知识库」？	Why 就近写在注释里——没人对准确性负责的知识库，只是搜索噪音
Q2 为什么我不让 Agent 写单元测试？	验证留给 human in the loop；Why 归注释，不把单测交给 Agent
Q3 为什么不给 Skill 做 Evaluation？	人对 skill 负责，比追求 eval 数值更重要；绝大多数 Harness 是债务，真资产是 CI hooks 与 Why 注释
Q4 为什么不用传统的 Git PR 工作流？	MR 不是审批——是确保负责人对变更负责的流程

负责接上，才把 Loop 接进工作台

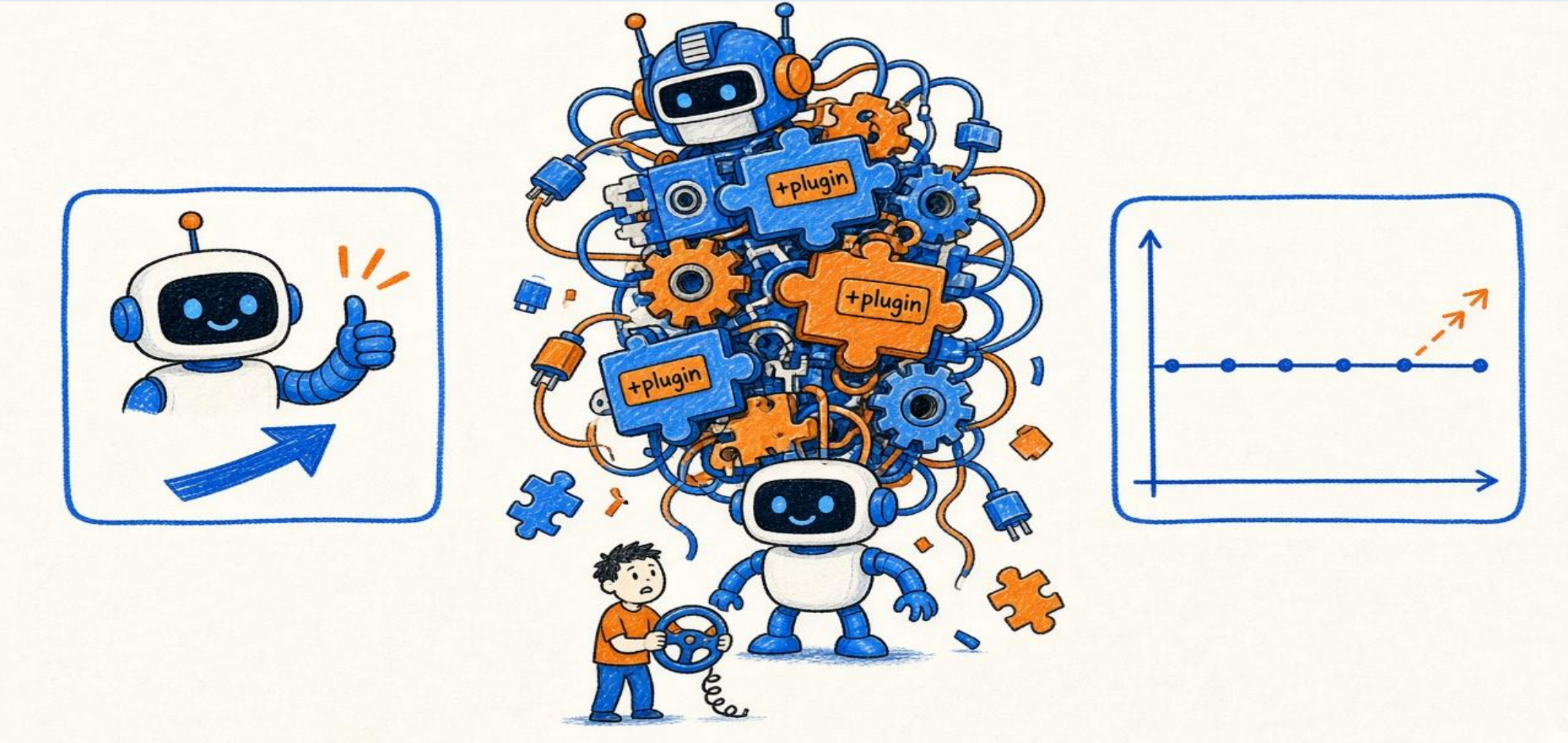
03

实践：Loop Harness

让策划自己开发需求

装了更多插件，Loop 就有了吗？

落点：Plugins and prompts——具象地说，use harness 常常就是加插件、加 prompt



给策划装更多面板，需求就自己跑起来了吗？

加法陷阱

插件、模型切换、质量面板、权限开关……

越堆越多。

策划注意力被工具界面吃掉，

需求仍跑不起来。

堆能力

减法界面

只留「这个需求要什么」：

目标、约束、可验收定义。

插件/模型/质量面板沉到 Harness

与研发侧，不上策划主路径。

只留要什么

Harness 加插件 · Loop 做减法

落点：Plugins and prompts——具象地说，use harness 常常就是加插件、加 prompt

「装了更多插件，loop 就有了」——往上堆插件，agent 像变形金刚：短期变好用，长期没有复利，装越多越 lose control。

Harness：加插件

把能力挂上去：工具、MCP、技能、上下文、权限……越加越多。

解决的是「能不能用上 Agent」。

加能力

Loop：为目标做减法

围绕目标收敛：输入、步骤、验收、纠错后的复利……越跑越稳。

解决的是「团队能不能持续交出去」。不用再反复投注意力或 token。

为目标做减法

转折：从追求 Token Maxxing 变成追求 ROI

Harness = Agent − Model ?

P8：Agent = Model + Harness → 反过来：减掉模型，剩下的叫什么？

Agent Harness

工具 · MCP · Skill · 上下文 · 权限

围着模型转——通用、可复用。

回答：Agent 能不能用上。

通用 · 围着模型

Business Harness（本体）

业务目标 · 验收 · 过线 · 责任边界
需求怎么流——贴着游戏业务。

回答：团队能不能持续交出去。

业务 · 围着过线

减掉模型剩下的，是通用工具，还是业务过线？
Agent Harness 回答「能不能用」· 把 Business Harness（本体）import 进 Build Loop

面向确定性的 Loop · 面向不确定性的 Loop

从追求 Token Maxxing 变成追求 ROI——先分清两种 Loop

面向确定性的 Loop

解决问题
提高 ROI

目标清楚、验收清楚——
把 token 换成可复利的价值。

确定性 · 价值转换

面向不确定性的 Loop

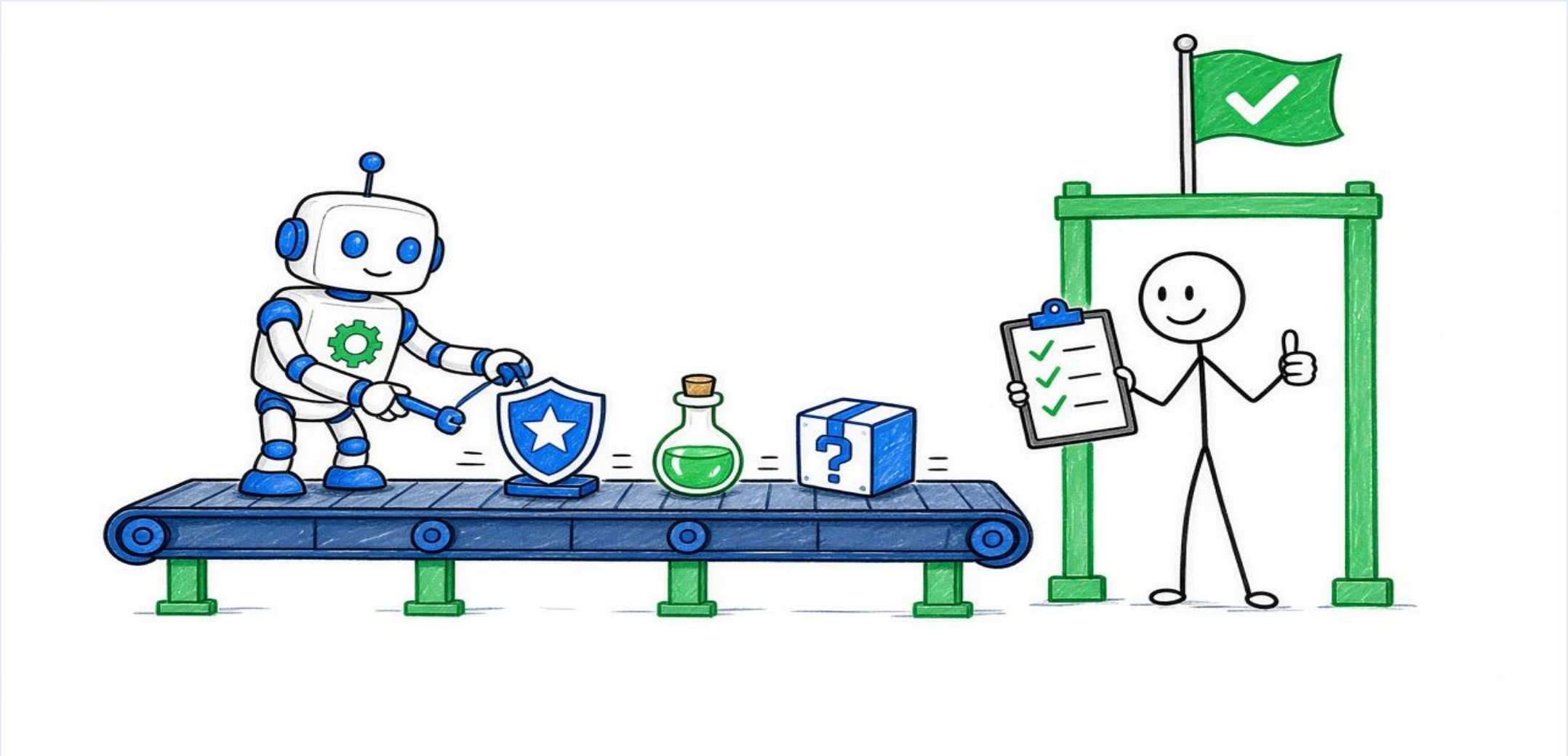
fail fast
勇于重做

方向未定、边界未清——
小圈试错，错了就换划界。

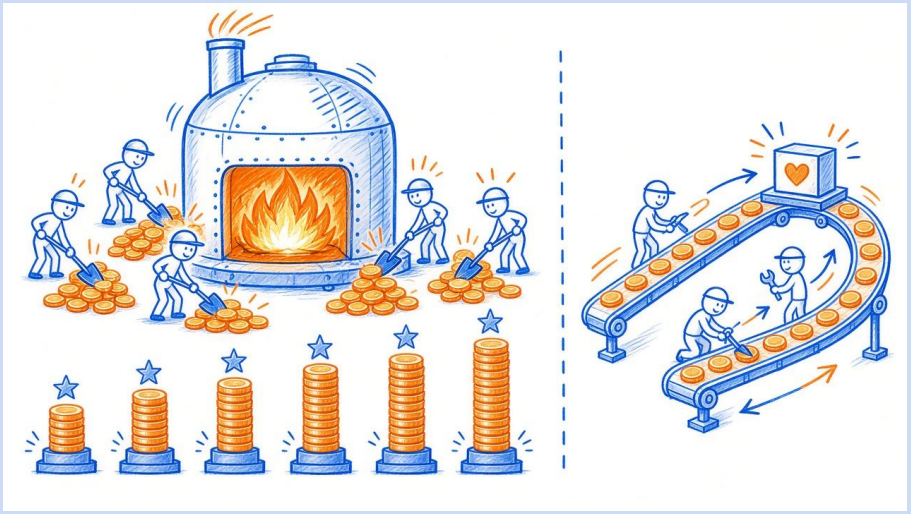
不确定性 · 快速重做

面向确定性的 Loop，解决问题、提高 ROI；面向不确定性的 Loop，fail fast，勇于重做

把验收交给策划，研发就能甩手了吗？



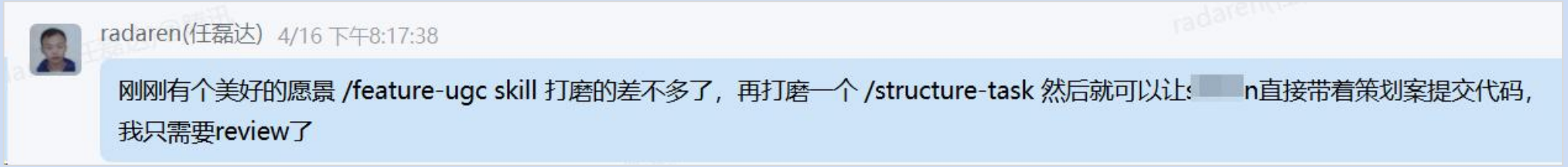
Token Maxxing 时代，比的是 token 谁花的多，哪个团队花的多



一个「美好的愿景」· 团队群聊截图

AI 时代，是不是应该让策划自己写代码？

愿景：/feature-ugc、/structure-task 两个 skill 打磨好 → 策划带着策划案直接提交代码，研发只 review



不要陷入 ROI 陷阱——找 token 变价值的转换 Loop | Harness→token · Loop→state

三件事：拆解 · 实现 · 自动化验收

策划在 loop 里出需求/实现；研发守过线——Loop Harness 只先钉这三件事。

策划在 loop 里

出需求、出实现：切开要什么、做出一版。

注意力放在「需求要什么」，
不放在质量背锅。

出需求 · 出实现

研发守住过线

守验收、守质量、守「这次能不能过」。

自动化验收是工程资产，
不是把测试转交给策划。

守过线 · 守质量

拆解

目标、边界、可验收切片，
不靠口头扯皮。

实现

Harness 里产出可跑一版，
不是再开「转研发」单。

自动化验收

红绿可复跑，Pass 写进 loop，
不靠「我觉得行」。

build loop = 架构工作
责任接上，才算 Loop；甩手只是换工种

Loop Harness 并不意味做数字同事

初期

让更多职能能用上模型。

用上模型

≠

长期

让每个人基于 Agent
对更多的事情负责。

不能负责，就还是 agent。

对更多事负责

吃得苦，霸得蛮，耐得烦

04

工作台让个人 Loop 到团队 Loop

Readability 变得比之前任何时候都重要

个体和互动高于流程和工具

让 Loop 转起来 · 让 Loop 转得起来

个人提效为什么变不成组织提效？——建 Loop 就是建连接；缺的是连接，不是 token

转起来 · 技术问题

要有人做功，或有 token 做功。

Demo / Harness 之后：
需求进、验收出，组织真开始转第一圈。

能转，不等于有人愿意接着转。

开第一圈

转得起来 · 机制问题

转得起来是另一套机制。

Work-based、小圈、Spec / 红节点、
可克隆工作台。

人愿意接着转，转了不空转——
connection 才落地。

接着转 · 不空转

个人提效 → 连接缺失 → 组织提效（可复利 / 可交接 / 可克隆）→ 判断：缺 Loop 的完美 Demo ≠ 业务稳定

事要一件一件做，饭要一口一口吃，multiagent 要一个一个 Loop
反馈让 Loop 转得起来，什么让 Loop 转得更快？

再开一个聊天窗，Loop 就有人接着转了吗？

Chat-based

聊天窗中心 → 易成垃圾场 / token 空转。

人被拖去新画布；会话断了，下一步丢了。

窗中心 · 停 token

Work-based

工作 / 状态中心 → 状态机 + 事件 + 目录 dsh。

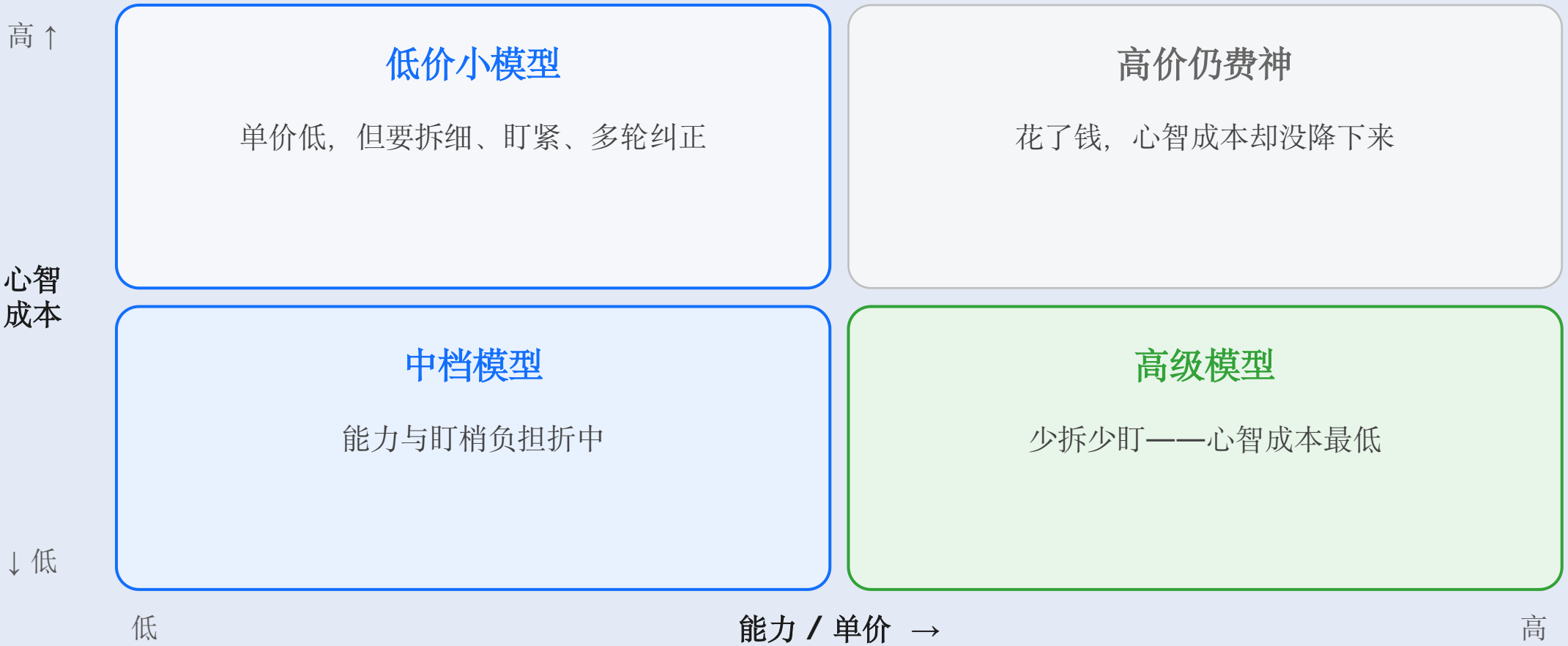
Agent 跟着代码走：在已有 IDE · Git · CI 上把体验做深。

状态中心 · 追 state

口令：少堆平台，深接工具

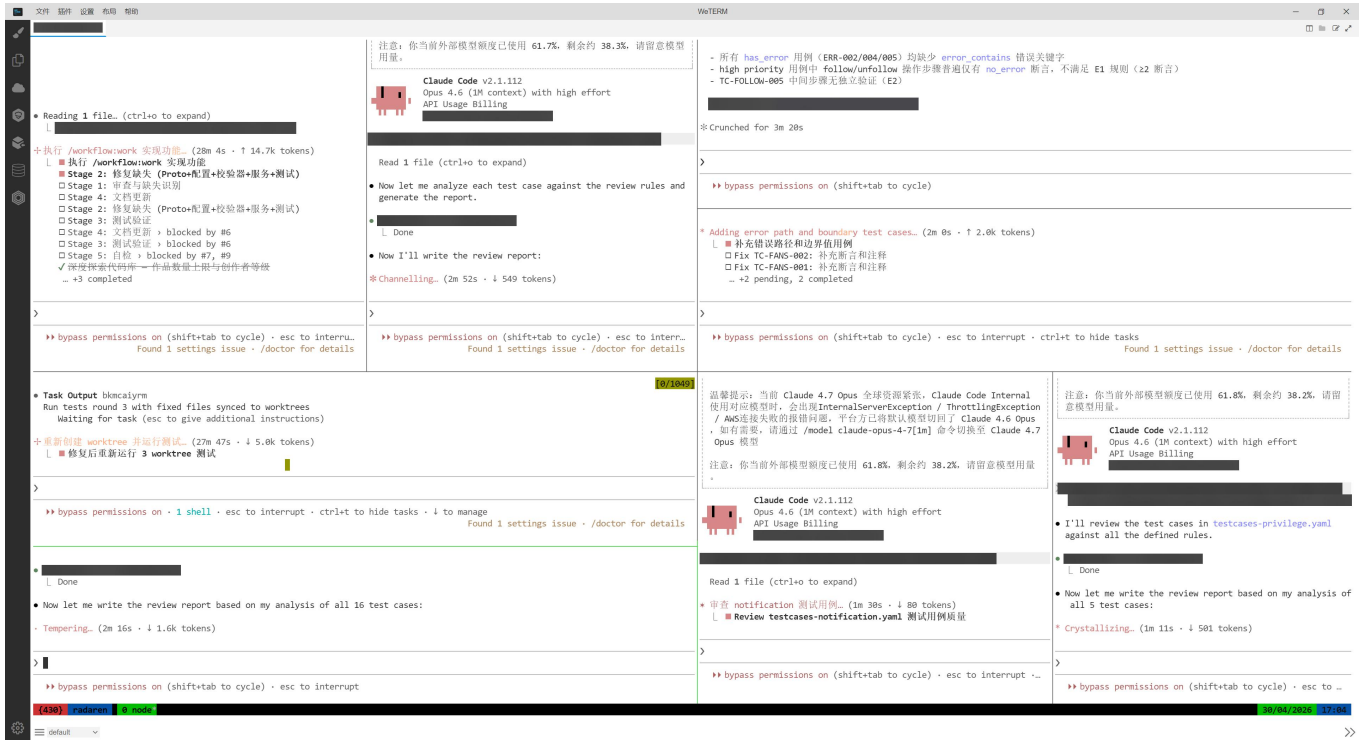
多 Agent 之前，先把单工具 Loop 接进日常路径 | 多平台 = 多上下文

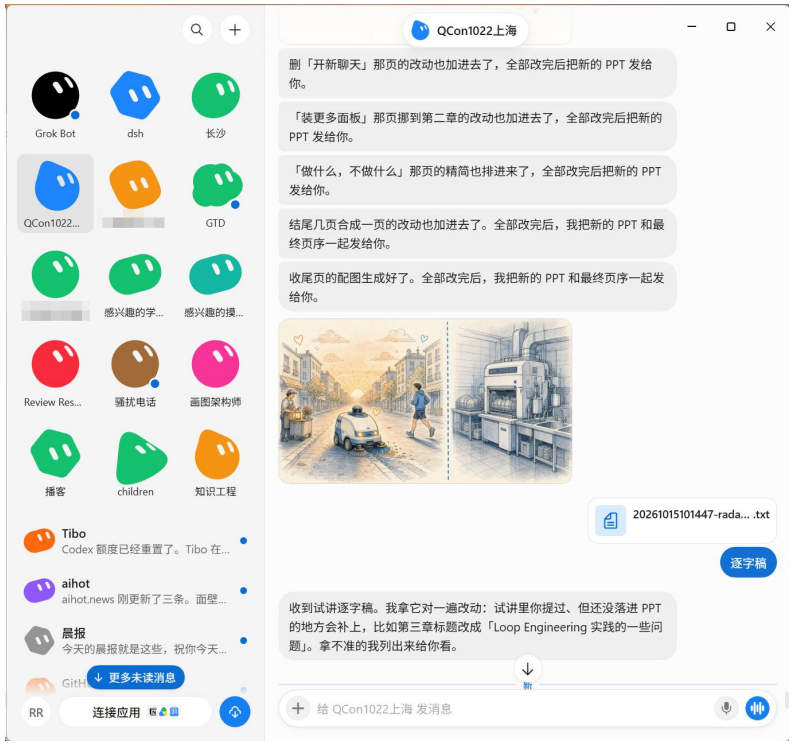
不同模型的 Token 价格到底差异在哪？



越高级的模型，用户的心智成本越低——价差在用户心智的投入

搭工作台之前：UI/UX · tmux





应该通过优异的 UI/UX 来 build 符合个人交互习惯的工作台

使用 DeepSeek Harness 搭建个人工作台

现在该验证哪一次构建。

防待办垃圾场：待办对准「哪一次构建要验」，不是又一张聊天待办。

日历/待办

构建是长任务；人离开，loop 还在转。

防会话过载：不把长构建绑死在前台聊天窗。

后台 tmux

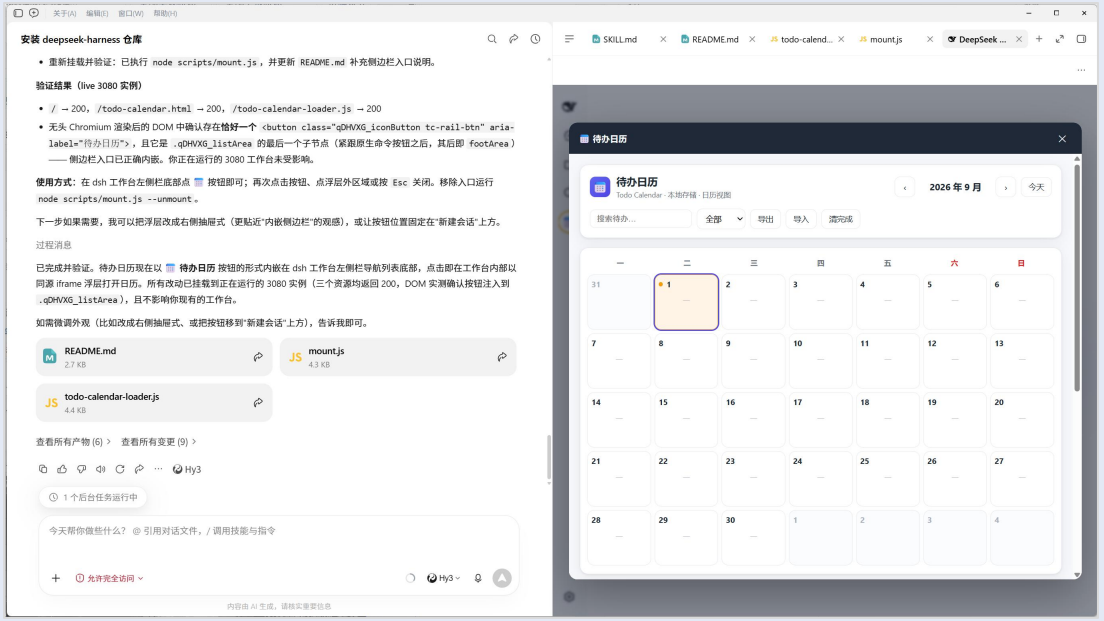
刚搭好的 dsh

插件...

基于 DeepSeek Harness：能力可复用 · 工作台可克隆

Harness ≈ Kafka：顺序/事件流则顺，当聊天窗乱捅则 thrash | 授人以渔：日常控制放在 Build Loop

日历：一句话，让 Agent 给工作台长出插件



创造模式里许愿

「做一个待办清单的插件，用日历的形式呈现我的待办列表」

↓ 先出独立视图——和 dsh 关系不大

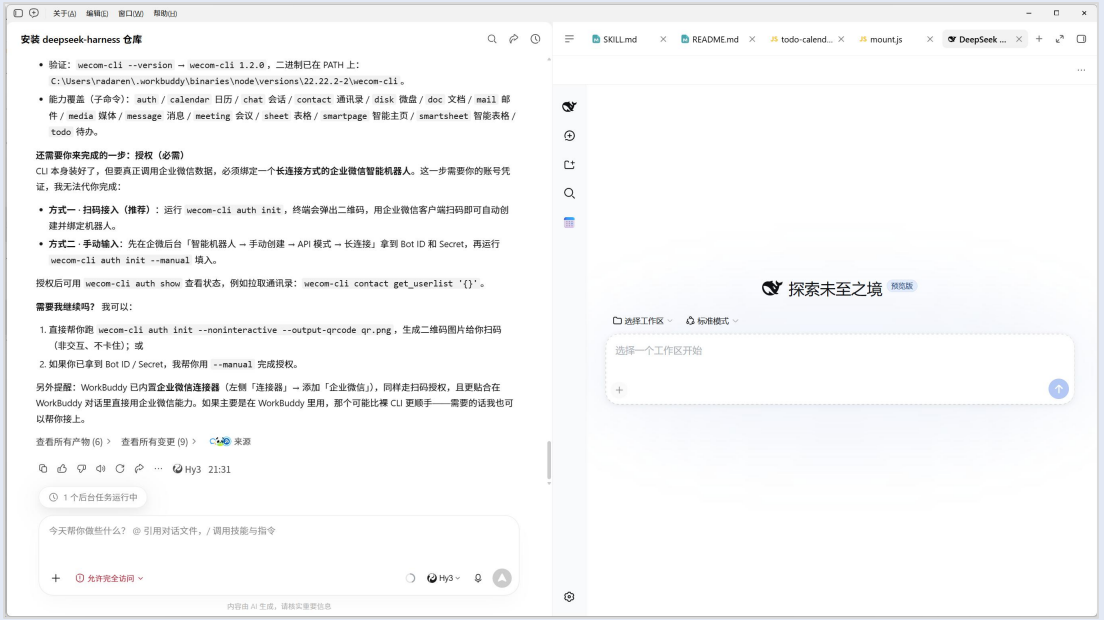
「作为侧边栏的一个跳转入口，内嵌到 dsh 侧边栏里面」

日历挂进左侧栏，点开即日历视图；
源码 + README 落盘：可复用、可克隆。

工作台不是装出来的，是用出来的

不报插件菜名：要什么，让 Agent 现写、现挂进 dsh 侧边栏

日历接上真实数据：让 Agent 自己去企业微信拉



1. 「帮我安装 wecom-cli」
2. 「生成二维码图片给我扫码」

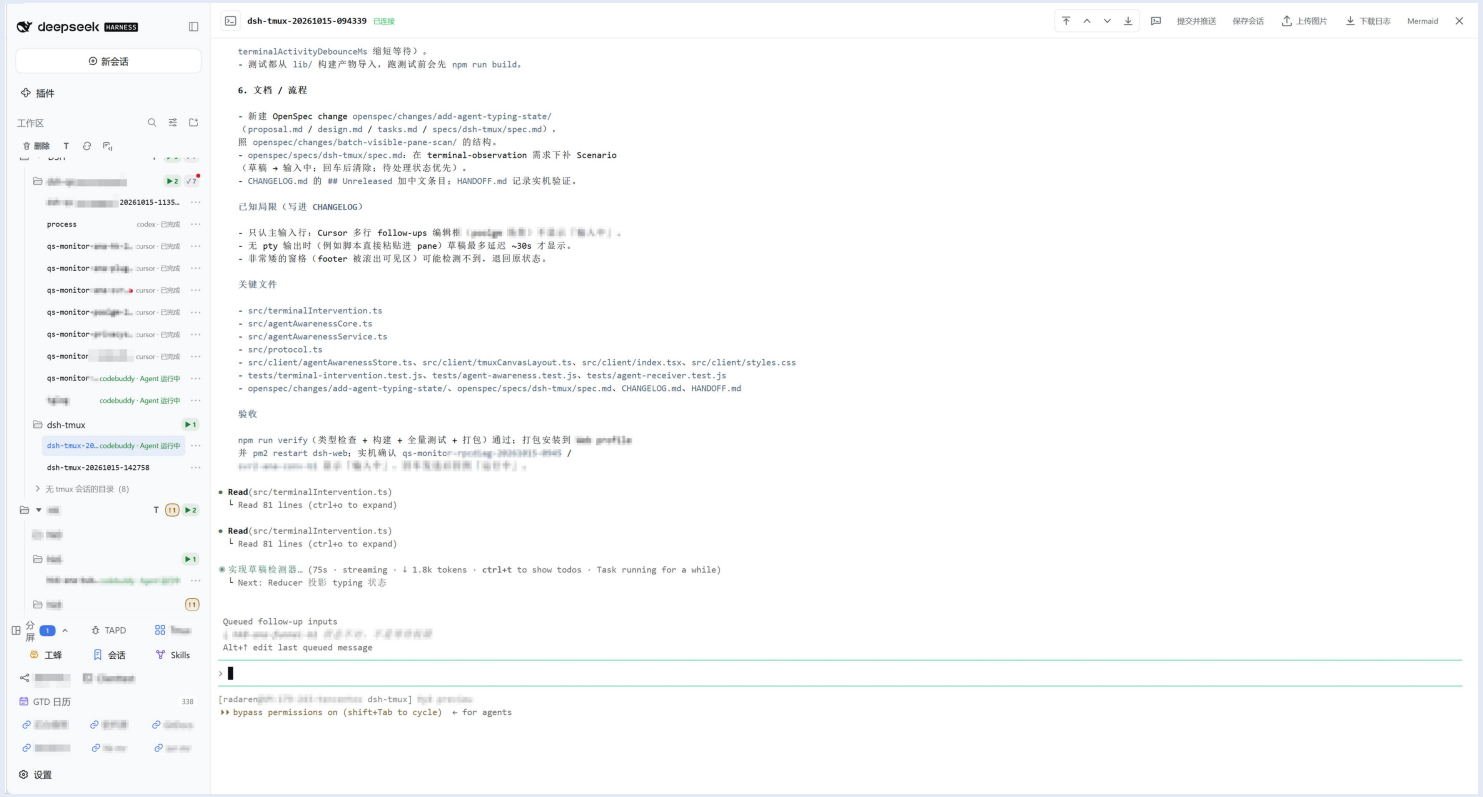
手机企业微信扫码，建一个拉日历的机器人

3. 「把企业微信里面的日历信息加载到上述日历里面」
4. 工作台里问：「接下来有哪些任务？」

日历对准「接下来该干什么」，不是又一张聊天待办

下一步：用版本管理（Git）存日历数据——state 可追溯、可交接

tmux：同一个套路接着长



后台 tmux：人离开，loop 还在转

构建是长任务，不绑死在前台聊天窗；tmux 开多少、跑到哪，可见。

同一个套路接着长

一句话许愿 → Agent 现写插件 → 挂进工作台 → 接真实数据。

能复用就复用

基于 DeepSeek Harness：能力可复用 · 工作台可克隆。

人离开 loop 还在转 · 长任务可见 · 同一个套路挂进工作台

个人工作台 → 管线工作台



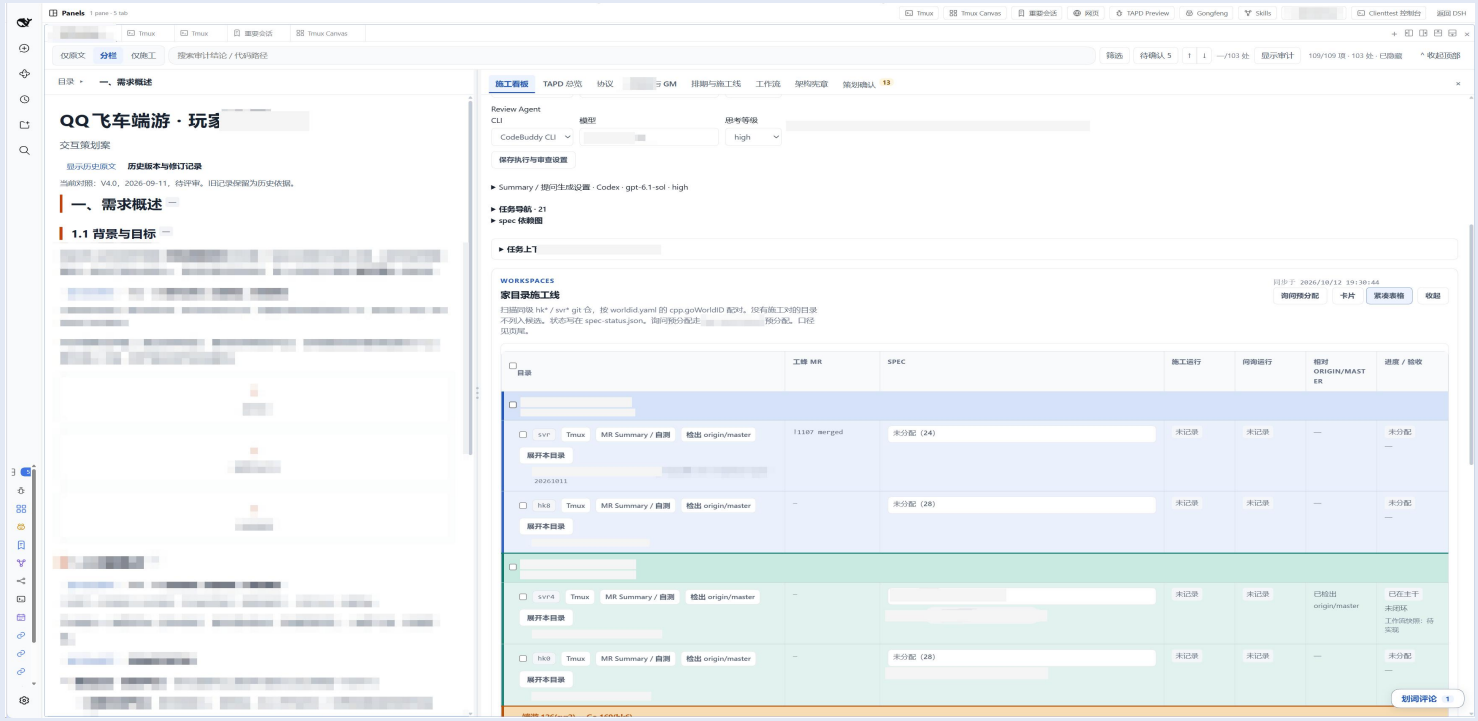
从个人工作台到管线工作台 · 拆解

个人工作台

· 侧边栏：日历 / tmux / Git
· 一句话许愿 → Agent 现写
· 自己先转起来——还停在「我的」一圈

我的一圈

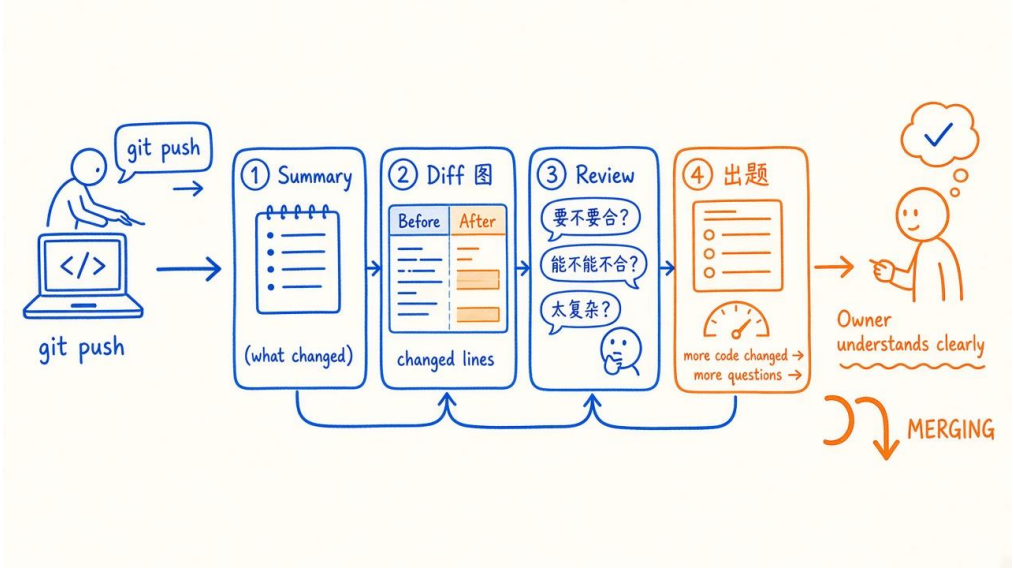
管线工作台 · 施工看板



策划 HTML 策划案 → 工作台：描述与 spec 双向跳转
策划改需求会生成新的 HTML；现在和策划一起迭代一个 dsh 插件，通过 git history 定位改动了什么，并定时提醒需求变更

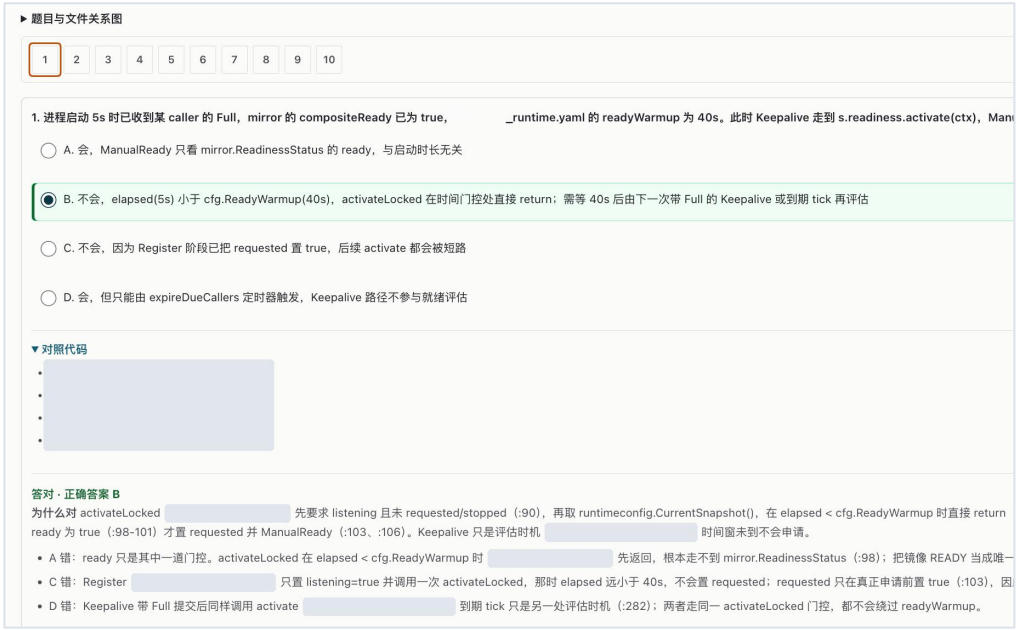
管线工作台：施工看板 · 工作流

工作流：Summary → Diff 图 → Review → 出题



AIGC 示意

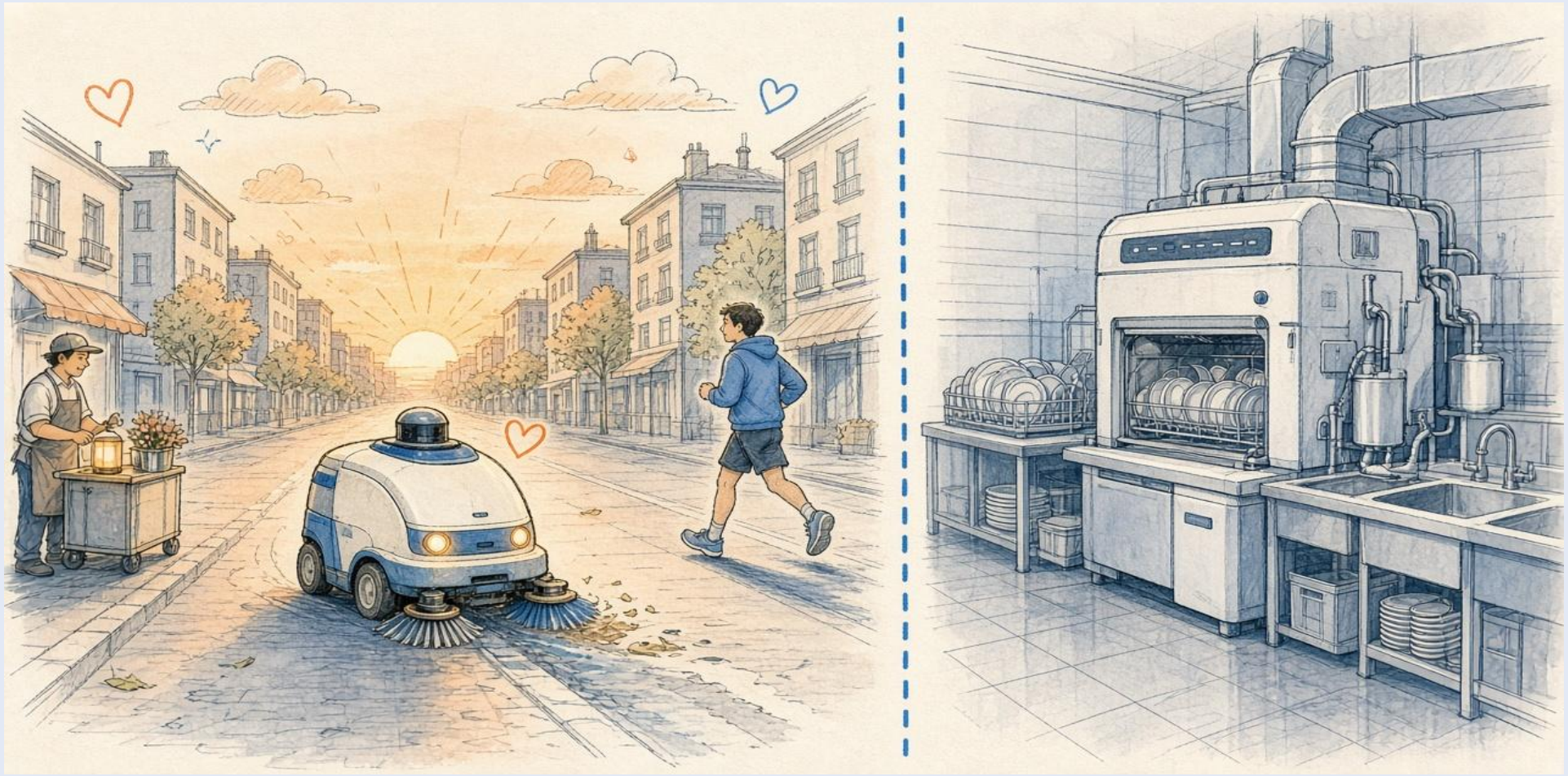
出题：改得越多题越多，确保 owner 认知清晰



真实出题页（已打码）

Rob Pike: "Readability is paramount."
那什么是 Readability?

1. 声明式接口 / 渐进式披露
2. 问题驱动 / 连接当前需求
3. 熟悉的配色和布局
4. Agent friendly
5. 信任：不需要读的文档是最好的文档
6. Simplicity is Complicated



凌晨五点的无人环卫车，比餐线上的自动洗碗机会显得更有温度

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software

延伸阅读

Agentic 研发转型过程中的 Loop Engineering

AICon 深圳 · 任磊达 · 2026-08-22

加我微信

微信扫码交流

任磊达 · 腾讯高级后台开发工程师

